

Batch: A2 Roll No.: 16010324044

Experiment / assignment / tutorial No. 1

Grade: AA / AB / BB / BC / CC / CD / DD

Signature of the Staff In-charge with date

TITLE: To create a new child process using fork system call.

Objectives: Programs using the Following system calls of Unix / Linux operating system: fork, exec, getpid, exit, wait.

OUTCOME: Student will be able to perform basic operating system tasks using command-line and will be able to write shell programming

fork()

- The fork system call is used to create a new process called *child* process.
 - The return value is 0 for a child process.
 - The return value is negative if process creation is unsuccessful.
 - For the parent process, return value is positive
- The child process is an exact copy of the parent process.
- Both the child and parent continue to execute the instructions following fork call.
- The child can start execution before the parent or vice-versa.

getpid() and getppid()

- The getpid system call returns process ID of the calling process
- The getppid system call returns parent process ID of the calling process

Algorithm

1. Declare a variable *x* to be shared by both child and parent.
2. Create a child process using fork system call.
3. If return value is -1 then

Print "Process creation unsuccessful" Terminate using exit system call.

4. If return value is 0 then

Print "Child process"

Print process id of the child using getpid system call Print value of *x*

Print process id of the parent using getppid system call

5. Otherwise

Print "Parent process"

K. J. Somaiya School of Engineering, Mumbai-77

Print process id of the parent using getpid system call
Print value of x
Print process id of the shell using getppid system call.
6. Stop

wait()

- The wait system call causes the parent process to be blocked until a child terminates.
- When a process terminates, the kernel notifies the parent by sending the SIGCHLD signal to the parent.
- Without wait, the parent may finish first leaving a *zombie* child, to be adopted by init process

Algorithm

1. Create a child process using fork system call.
2. If return value is -1 then
 - a. Print "Process creation unsuccessful"
3. Terminate using exit system call.
4. If return value is > 0 then
 - a. Suspend parent process until child completes using wait system call
 - b. Print "Parent starts"
 - c. Print even numbers from 0–10
 - d. Print "Parent ends"
5. If return value is 0 then
 - a. Print "Child starts"
 - . Print odd numbers from 0–10
 - c. Print "Child ends"
6. Stop

Part 1:-

```
dmspe@D-Laptop:~$ ps
  PID TTY          TIME CMD
  409 pts/0        00:00:00 bash
  622 pts/0        00:00:00 ps
```

```
dmspe@D-Laptop:~$ ps -e
  PID TTY          TIME CMD
    1 ?           00:00:00 systemd
    2 hvc0        00:00:00 init-systemd(Ub
    7 hvc0        00:00:00 init
   43 ?           00:00:00 systemd-journal
   93 ?           00:00:00 systemd-udevd
  108 ?           00:00:00 systemd-resolve
  109 ?           00:00:00 systemd-timesyn
  150 ?           00:00:00 avahi-daemon
  152 ?           00:00:00 cron
  153 ?           00:00:00 dbus-daemon
  163 ?           00:00:00 lttng-sessiond
  173 ?           00:00:00 polkitd
  183 ?           00:00:00 lttng-runas
  185 ?           00:00:00 systemd-logind
  195 ?           00:00:00 udisksd
  202 ?           00:00:00 wsl-pro-service
  212 ?           00:00:00 rsyslogd
  225 ?           00:00:00 avahi-daemon
  235 ?           00:00:00 wpa_supplicant
  243 ?           00:00:00 unattended-upgr
  247 tty1        00:00:00 agetty
  342 ?           00:00:00 systemd-timedat
  407 ?           00:00:00 SessionLeader
  408 ?           00:00:00 Relay(409)
  409 pts/0        00:00:00 bash
  410 pts/1        00:00:00 login
  458 ?           00:00:00 systemd
  459 ?           00:00:00 (sd-pam)
  488 pts/1        00:00:00 bash
  623 pts/0        00:00:00 ps
```

```
dmspe@D-Laptop:~$ ps -f
UID          PID    PPID  C STIME TTY          TIME CMD
dmspe        409      408  0 18:16 pts/0        00:00:00 -bash
dmspe        624      409  0 18:16 pts/0        00:00:00 ps -f
```

```
dmspe@D-Laptop:~$ ps -ef
UID          PID    PPID  C STIME TTY          TIME CMD
root           1         0  2 18:16 ?            00:00:00 /sbin/init
root           2         1  0 18:16 hvc0        00:00:00 /init
root           7         2  0 18:16 hvc0        00:00:00 plan9 --control-sc
root          43         1  0 18:16 ?            00:00:00 /usr/lib/systemd/s
root          93         1  0 18:16 ?            00:00:00 /usr/lib/systemd/s
systemd+     108         1  0 18:16 ?            00:00:00 /usr/lib/systemd/s
systemd+     109         1  0 18:16 ?            00:00:00 /usr/lib/systemd/s
avahi        150         1  0 18:16 ?            00:00:00 avahi-daemon: runn
root        152         1  0 18:16 ?            00:00:00 /usr/sbin/cron -f
message+    153         1  0 18:16 ?            00:00:00 @dbus-daemon --sys
root        163         1  0 18:16 ?            00:00:00 /usr/bin/lttng-ses
polkitd     173         1  0 18:16 ?            00:00:00 /usr/lib/polkit-1/
root        183        163  0 18:16 ?            00:00:00 /usr/bin/lttng-ses
root        185         1  0 18:16 ?            00:00:00 /usr/lib/systemd/s
root        195         1  0 18:16 ?            00:00:00 /usr/libexec/udisk
root        202         1  0 18:16 ?            00:00:00 /usr/libexec/wsl-p
syslog      212         1  0 18:16 ?            00:00:00 /usr/sbin/rsyslogd
avahi       225        150  0 18:16 ?            00:00:00 avahi-daemon: chro
root        235         1  0 18:16 ?            00:00:00 /usr/sbin/wpa_supp
root        243         1  0 18:16 ?            00:00:00 /usr/bin/python3 /
root        247         1  0 18:16 tty1        00:00:00 /sbin/agetty -o -p
root        407         2  0 18:16 ?            00:00:00 /init
root        408        407  0 18:16 ?            00:00:00 /init
dmspe       409        408  0 18:16 pts/0        00:00:00 -bash
root        410         2  0 18:16 pts/1        00:00:00 /bin/login -f
dmspe       458         1  0 18:16 ?            00:00:00 /usr/lib/systemd/s
dmspe       459        458  0 18:16 ?            00:00:00 (sd-pam)
dmspe       488        410  0 18:16 pts/1        00:00:00 -bash
dmspe       628        409  0 18:16 pts/0        00:00:00 ps -ef
```

K. J. Somaiya School of Engineering, Mumbai-77

```
dmspe@D-Laptop:~$ ps aux
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  2.1  0.1  21820 13272 ?        Ss   18:16   0:00 /sbin/init
root         2  0.0  0.0   3180  2208 hvc0     Sl+  18:16   0:00 /init
root         7  0.0  0.0   3180  2044 hvc0     Sl+  18:16   0:00 plan9 --control-s
root        43  0.5  0.2  42244 16268 ?        S<s  18:16   0:00 /usr/lib/systemd/
root        93  0.5  0.0  26596  7600 ?        Ss   18:16   0:00 /usr/lib/systemd/
systemd+  108  0.3  0.1  21344 13208 ?        Ss   18:16   0:00 /usr/lib/systemd/
systemd+  109  0.1  0.0  91060  7956 ?        Ssl  18:16   0:00 /usr/lib/systemd/
avahi     150  0.0  0.0   8696  4592 ?        Ss   18:16   0:00 avahi-daemon: run
root     152  0.0  0.0   4244  2680 ?        Ss   18:16   0:00 /usr/sbin/cron -f
message+  153  0.2  0.0   9772  5516 ?        Ss   18:16   0:00 @dbus-daemon --sy
root     163  0.1  0.1 821208 11412 ?        Ssl  18:16   0:00 /usr/bin/lttng-se
polkitd   173  0.3  0.1 308172  8204 ?        Ssl  18:16   0:00 /usr/lib/polkit-1
root     183  0.0  0.0  49568  2228 ?        S    18:16   0:00 /usr/bin/lttng-se
root     185  0.2  0.1  17984  8888 ?        Ss   18:16   0:00 /usr/lib/systemd/
root     195  0.3  0.1 468968 13840 ?        Ssl  18:16   0:00 /usr/libexec/udis
root     202  0.3  0.1 1756628 13580 ?        Ssl  18:16   0:00 /usr/libexec/wsl-
syslog    212  0.2  0.0 222516  5948 ?        Ssl  18:16   0:00 /usr/sbin/rsyslog
avahi     225  0.0  0.0   8432  1528 ?        S    18:16   0:00 avahi-daemon: chr
root     235  0.0  0.0  17392  6372 ?        Ss   18:16   0:00 /usr/sbin/wpa_sup
root     243  0.6  0.3 112848 25232 ?        Ssl  18:16   0:00 /usr/bin/python3
root     247  0.0  0.0   3124  1976 tty1     Ss+  18:16   0:00 /sbin/agetty -o -
root     407  0.0  0.0   3184  1112 ?        Ss   18:16   0:00 /init
root     408  0.0  0.0   3200  1252 ?        S    18:16   0:00 /init
dmspe     409  0.3  0.0   6264  5552 pts/0    Ss   18:16   0:00 -bash
root     410  0.0  0.0   6864  4676 pts/1    Ss   18:16   0:00 /bin/login -f
dmspe     458  0.5  0.1  20192 11388 ?        Ss   18:16   0:00 /usr/lib/systemd/
dmspe     459  0.0  0.0  21200  3604 ?        S    18:16   0:00 (sd-pam)
dmspe     488  0.1  0.0   6200  5484 pts/1    S+   18:16   0:00 -bash
dmspe     632  0.0  0.0   8340  4356 pts/0    R+   18:17   0:00 ps aux
```

```
dmspe@D-Laptop:~$ ps -u
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
dmspe     409  0.2  0.0   6264  5552 pts/0    Ss   18:16   0:00 -bash
dmspe     488  0.1  0.0   6200  5484 pts/1    S+   18:16   0:00 -bash
dmspe     635  0.0  0.0   8340  4376 pts/0    R+   18:17   0:00 ps -u
```

```
dmspe@D-Laptop:~$ ps -l
F S    UID      PID      PPID    C  PRI   NI ADDR SZ WCHAN  TTY      TIME CMD
4 S    1000      409      408    0   80    0  -  1566 do_wai pts/0    00:00:00 bash
0 R    1000      636      409    0   80    0  -  2093 -      pts/0    00:00:00 ps
```

```
dmspe@D-Laptop:~$ ps -A
  PID TTY          TIME CMD
    1 ?           00:00:00 systemd
    2 hvc0        00:00:00 init-systemd(Ub
    7 hvc0        00:00:00 init
   43 ?           00:00:00 systemd-journal
   93 ?           00:00:00 systemd-udevd
  108 ?           00:00:00 systemd-resolve
  109 ?           00:00:00 systemd-timesyn
  150 ?           00:00:00 avahi-daemon
  152 ?           00:00:00 cron
  153 ?           00:00:00 dbus-daemon
  163 ?           00:00:00 lttng-sessiond
  173 ?           00:00:00 polkitd
  183 ?           00:00:00 lttng-runas
  185 ?           00:00:00 systemd-logind
  195 ?           00:00:00 udisksd
  202 ?           00:00:00 wsl-pro-service
  212 ?           00:00:00 rsyslogd
  225 ?           00:00:00 avahi-daemon
  235 ?           00:00:00 wpa_supplicant
  243 ?           00:00:00 unattended-upgr
  247 tty1        00:00:00 agetty
  407 ?           00:00:00 SessionLeader
  408 ?           00:00:00 Relay(409)
  409 pts/0        00:00:00 bash
  410 pts/1        00:00:00 login
  458 ?           00:00:00 systemd
  459 ?           00:00:00 (sd-pam)
  488 pts/1        00:00:00 bash
  637 pts/0        00:00:00 ps
```

Part 2:

K. J. Somaiya School of Engineering, Mumbai-77

Write a Program for creating process using System call (E.g fork())
Create a child process. Display the details about that process using
getpid and getppid functions. In a child process, Open the file using file
system calls and read the contents and display.

- Use any online C editor/IDE for the above program.
- Explore the working of the fork() system call and document it with examples
- Explore the working of the functions getpid() , getppid()
- Corresponding Header files needed.
- Use the Value returned by the fork system call , to create if,elseif and else block,
If for system call failure, elseif for child block, else for parent process block.
Further perform assigned task as per question.
- Save the screenshot of the code and output of the above program.

Program Code and Screenshot

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<fcntl.h>
#include<sys/types.h>
#include<sys/wait.h>
```

```
int main()
{
    pid_t pid;
    int fd;
    char buffer[100];

    pid = fork();

    if(pid < 0)
```

```
{  
    printf("Process creation unsuccessful\n");  
    exit(1);  
}
```

```
else if(pid == 0)
```

```
{  
    printf("\n----- Child Process ----- \n");  
  
    printf("Child PID : %d\n", getpid());  
    printf("Parent PID: %d\n", getppid());  
  
    fd = open("sample.txt", O_RDONLY);
```

```
    if(fd < 0)
```

```
{  
        printf("Unable to open file.\n");  
        exit(1);  
    }
```

```
    int n = read(fd, buffer, sizeof(buffer)-1);  
    buffer[n] = '\0';
```

```
    printf("\nContents of File:\n");  
    printf("%s\n", buffer);
```

```
    close(fd);
```

```
}
```

```
else
```

```
{  
    wait(NULL);
```

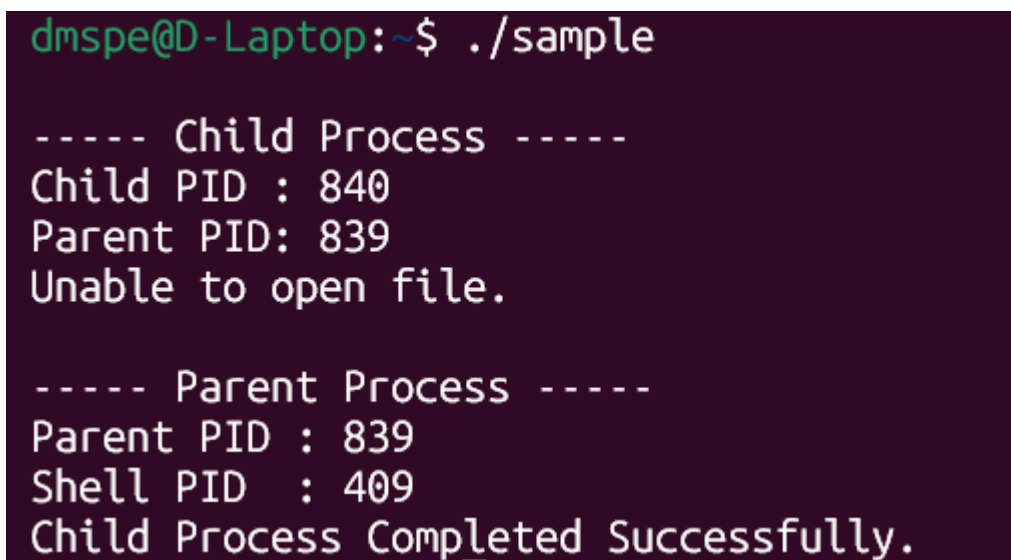


```
printf("\n----- Parent Process ----- \n");

printf("Parent PID : %d\n", getpid());
printf("Shell PID : %d\n", getppid());

printf("Child Process Completed Successfully.\n");
}

return 0;
}
```



```
dmspe@D-Laptop:~$ ./sample

----- Child Process -----
Child PID : 840
Parent PID: 839
Unable to open file.

----- Parent Process -----
Parent PID : 839
Shell PID : 409
Child Process Completed Successfully.
```

Conclusion

In this experiment, I successfully demonstrated the creation of a child process using the `fork()` system call. The parent and child processes were identified using `getpid()` and `getppid()`. The child process successfully opened and read a file using file system calls, while the parent synchronized using `wait()`. Thus, the objectives of understanding process creation, process identification, and synchronization in Linux were achieved.

Signature of faculty in-charge